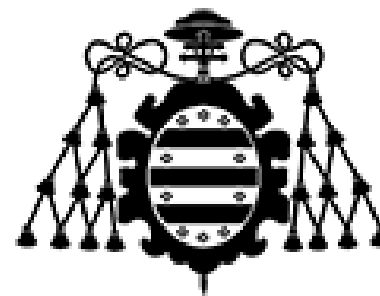




Escuela de
Ingeniería
Informática
Universidad de Oviedo



Universidad de
Oviedo

Trabajo Grupal ASSW

Manuel García Baldó – UO295497

Raúl Antuña Suárez – UO294202

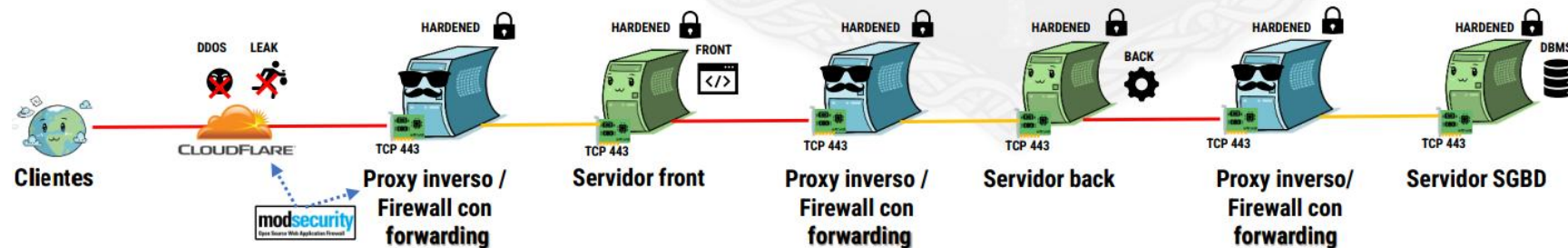
Índice

1. Contexto Inicial
2. Objetivo
3. Solución planteada
4. Implementación
5. Despliegue
6. Resultados
7. Problemas encontrados
8. Conclusiones



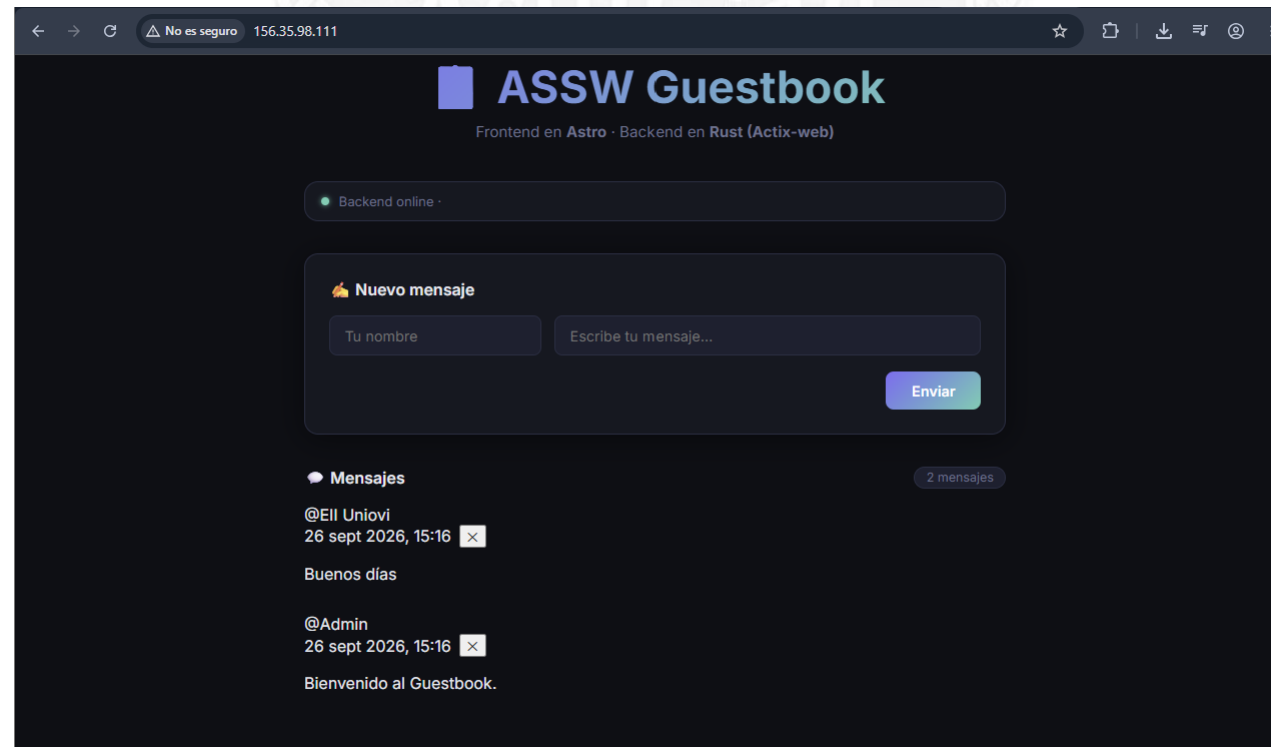
Contexto inicial

- El proyecto simula un proveedor de infraestructura Cloud y DevSecOps, enfocado en el despliegue automatizado y seguro de aplicaciones web con capas de frontend y backend aisladas.



Objetivo

- Desplegar la aplicación web en una infraestructura segura, de manera automatizada, para mostrar una web funcional con frontend y backend.

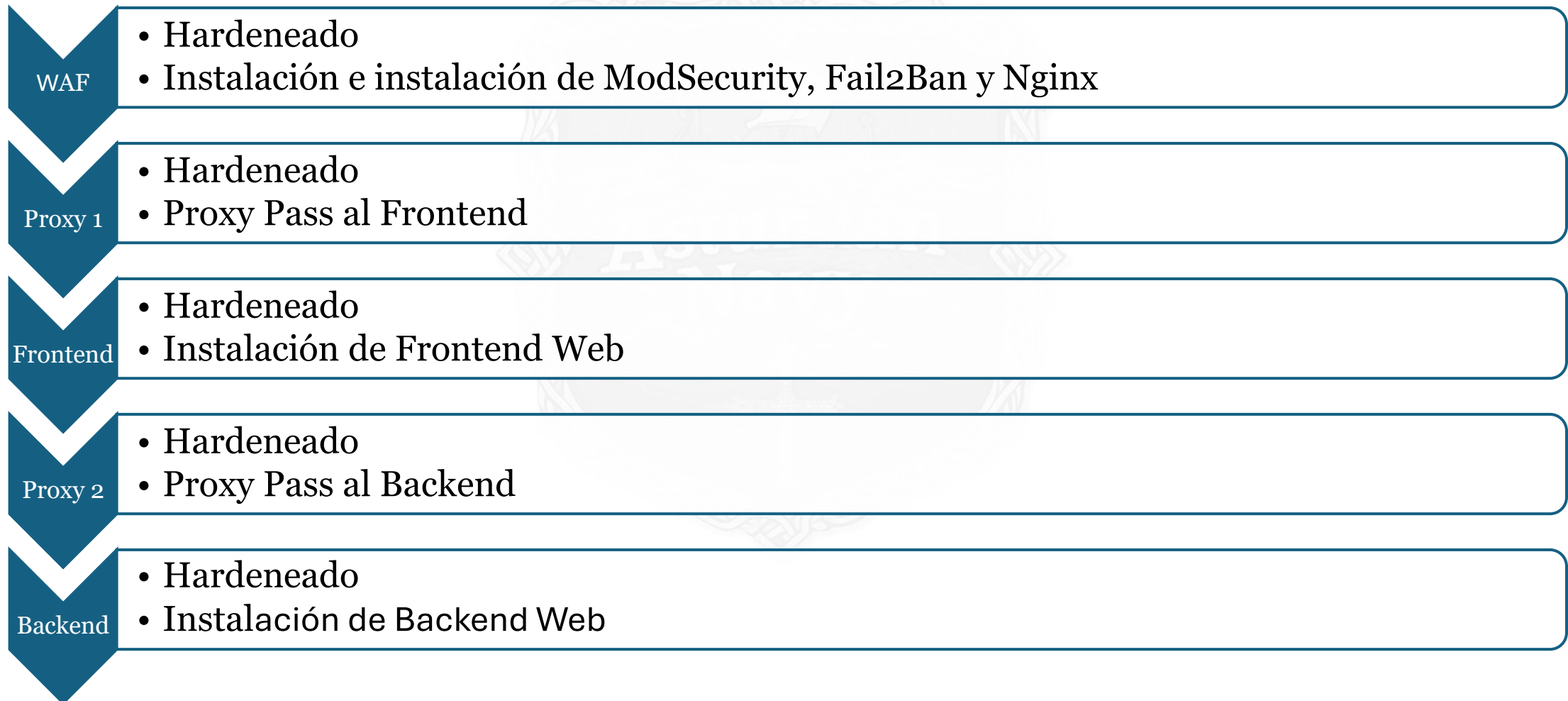


Solución planteada

- Creación de infraestructura como código por Terraform.
- Aprovisionamiento de la misma mediante Ansible.
- Uso del servidor web Nginx para manejar las diferentes capas.



Solución planteada



Implementación > Terraform

- Definición de la infraestructura en **main.tf**
- Definición de script a ejecutar en **locals.tf** (en nuestro caso realmente no hace nada, solo imprime “Instancia inicializada” en cada máquina.
- Definición de redes a crear en **network.tf**
- Dentro de **providers.tf** definimos el provider de OpenStack. Hemos dejado unas definiciones de local y external también, ya que hemos estado haciendo pruebas obteniendo IP flotantes de forma automática, sin éxito.

Implementación > Terraform

- Red privada aislada: Creación de red y subred interna (10.0.10.0/24).
- Enrutamiento: Router conectado a la red externa (public) para salida a Internet.
- Grupos de Seguridad (Firewall):
 - WAF (Público): Solo puertos 22 (SSH), 80 (HTTP) y 443 (HTTPS) abiertos.
 - Interno (Privado): Tráfico restringido exclusivamente dentro de la red local.

Implementación > Terraform

- Instancias Ubuntu en 2 lotes (lo teníamos planificado con un bucle `for_each`, pero lo cambiamos por `timeouts`):
 - Lote 1: WAF perimetral y Frontend.
 - Lote 2: Backend y dos Proxies internos.
- Punto de entrada único: Asignación de IP Flotante pública solo al WAF.
- Puente con Ansible: Generación automática del inventario (`inventory.ini`) usando el WAF como Bastion / ProxyJump.

Implementación > Ansible

- Hardening base (todas las MVs):
 - Actualización de repositorios y paquetes del sistema.
 - Firewall local UFW activo (Deny por defecto, solo puertos esenciales).
- WAF perimetral:
 - Nginx + ModSecurity activo como cortafuegos web de aplicaciones.
 - Fail2Ban para mitigar ataques de fuerza bruta.
 - Proxy inverso: enruta `/` al Frontend y `/api/` al Backend.

Implementación > Ansible

- Backend (Rust): github.com/manugbd/ASSW_BACK
 - Instalación de toolchain Rust (cargo/rustup).
 - Clonado de repositorio y compilación en release.
 - Ejecución como servicio demonio systemd (puerto 8080).
- Frontend (SPA Web): github.com/manugbd/ASSW_FRONT
 - Entorno Node.js 20 y clonado del repositorio.
 - Compilación estática del proyecto (npm run build).
 - Nginx configurado para servir la web y gestionar rutas SPA.

Uso del WAF (Web Application Firewall)

- El usuario solo interactúa con la IP del WAF.
- Enrutamiento por contexto:
 - Reenvío interno a la IP privada del Frontend (puerto 80).
 - `/api/` Reenvío interno a la IP privada del Backend (puerto 8080).
- ModSecurity filtra ataques en ambas rutas antes de alcanzar las máquinas internas.

Despliegue de la aplicación web > Frontend

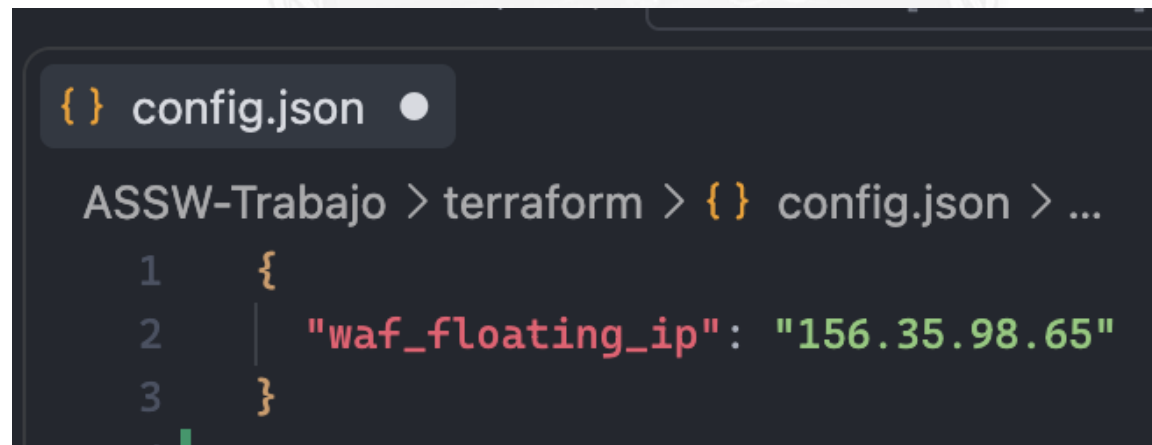
- Build de producción:
 - Aprovisionamiento del runtime Node.js 20 LTS.
 - Compilación de ficheros estáticos (npm run build).
- Servidor Web Nginx dedicado:
 - Servido de archivos desde `/var/www/assw_front`.
 - Soporte para enrutamiento SPA (`try_files $uri $uri/ /index.html`) para evitar errores 404 al navegar.

Despliegue de la aplicación web > Backend

- Instalación desatendida de rustup y dependencias C/SSL.
- Acciones durante el despliegue.
 - Clonado del repositorio desde GitHub.
 - Compilación nativa en modo optimizado (cargo build --release).
- Uso de systemd.
 - Binario gestionado como servicio del sistema en el puerto 8080.
 - Política de auto-reinicio ante fallos **(Restart=on-failure)**.

Despliegue del trabajo

- **IMPORTANTE:** Se debe definir una IP flotante disponible en el archivo `config.json` de la carpeta `terraform`.
- Hemos intentado recuperar dicha clave de forma dinámica, pero parece ser que el proveedor no dispone de un método directo de hacerlo.



The screenshot shows a code editor with a file named `config.json` open. The breadcrumb navigation at the top reads `ASSW-Trabajo > terraform > {} config.json > ...`. The code content is as follows:

```
1  {
2    "waf_floating_ip": "156.35.98.65"
3  }
```

Despliegue del trabajo

- Uso del script **deploy.sh**
 - Para usarlo se requiere pasar como segundo argumento la clave del fichero OpenRC para cargar las variables de entorno.
 - Opcionalmente se le puede pasar un argumento **--key** que contenga la clave pública a utilizar para conectar Ansible a las máquinas. Si no se especifica, se genera un par y se usa para la conexión.

```
● uo294202@uniovi:~/ASSW-Trabajo$ ./deploy.sh -h
Uso: ./deploy.sh <archivo-openrc.sh> [--key <ruta-clave-ssh>]
Ejemplo: ./deploy.sh ~/admin-openrc.sh --key ~/.ssh/deploy_key
❖❖ uo294202@uniovi:~/ASSW-Trabajo$ █
```

Resultados

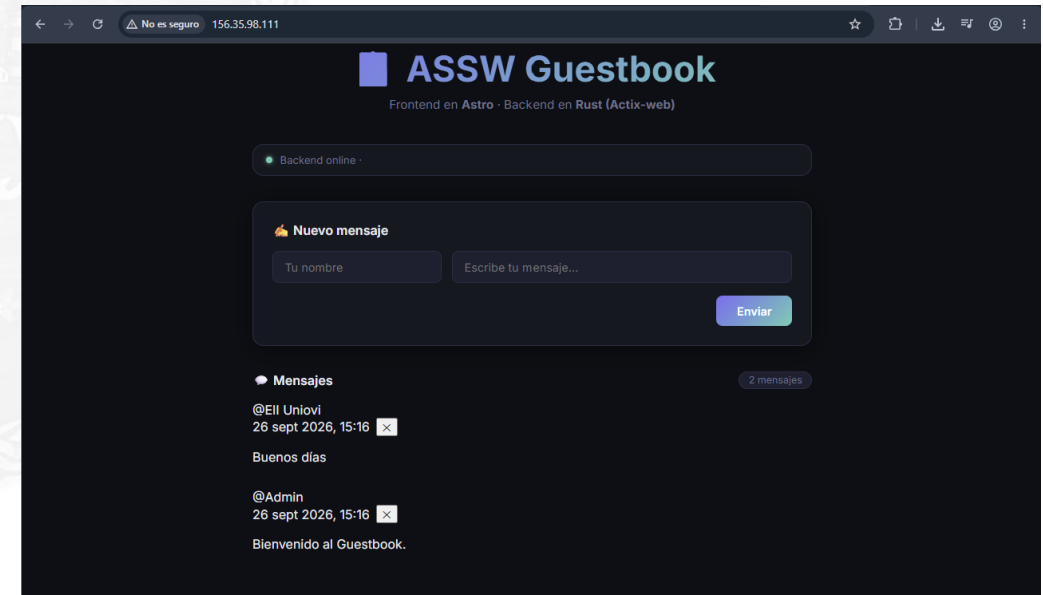
- Hemos conseguido desplegar la infraestructura y comprobar que funciona correctamente.

openstack. UO295497_project ▼

Mostrando 6 artículos

☐	Nombre de la instancia ▲	Nombre de la imagen	Dirección IP	Sabor	Par de claves	Estado
☒	backend	Ubuntu-26.04	10.0.10.88	ephym_small_2	infra-segura-key	Activa
☒	frontend	Ubuntu-26.04	10.0.10.71	ephym_small_2	infra-segura-key	Activa
☒	proxy-interno-1	Ubuntu-26.04	10.0.10.209	ephym_small_2	infra-segura-key	Activa
☒	proxy-interno-2	Ubuntu-26.04	10.0.10.174	ephym_small_2	infra-segura-key	Activa
☐	UO295497-ubuntu-26.04-nueva	Ubuntu-26.04	192.168.1.212, 156.35.98.112	small_2	-	Activa
☒	waf-perimetral	Ubuntu-26.04	10.0.10.180, 156.35.98.111	ephym_small_2	infra-segura-key	Activa

Mostrando 6 artículos

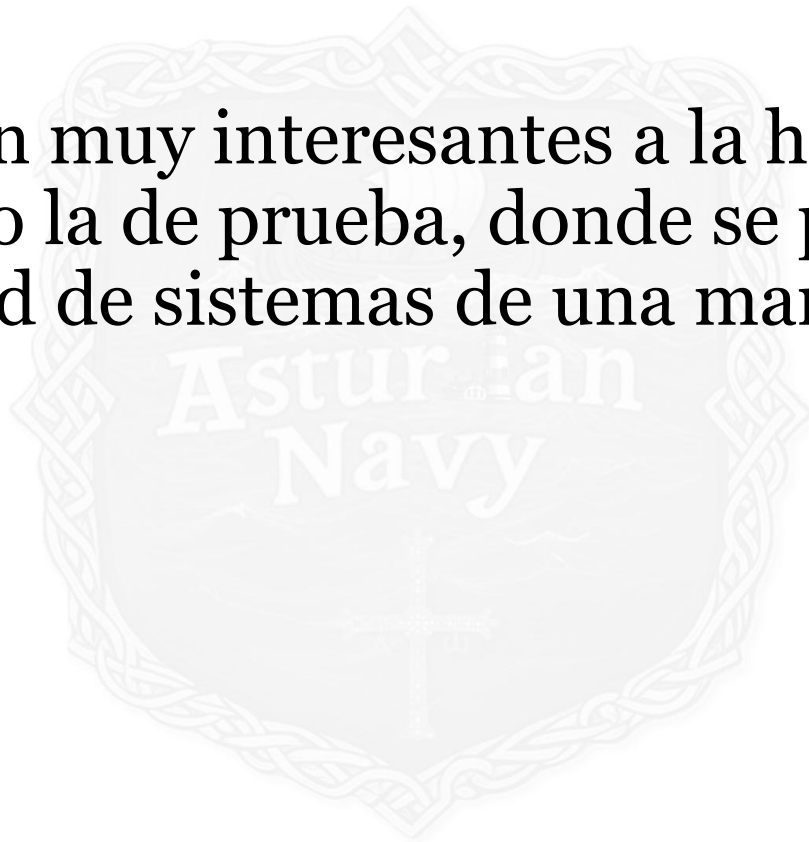


Problemas encontrados

- Infraestructura de OpenStack
 - Hemos tenido dificultades a la hora de desplegar y probar las infraestructuras.
 - Elevada duración de despliegue de Terraform.
 - Esto muchas veces generaba timeouts en Ansible y al relanzar nos borraba y planificaba la máquina de nuevo.
- Planificaciones con Ansible
 - Al ser algo nuevo para nosotros, tuvimos que buscar información sobre la estructura del yaml y los estados disponibles para cada task.

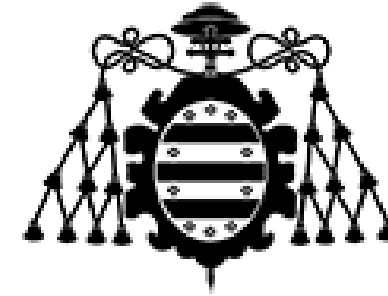
Conclusiones

Estás tecnologías son muy interesantes a la hora de controlar una infraestructura como la de prueba, donde se pueden llegar a automatizar infinitud de sistemas de una manera sencilla, segura y muy extensible.





Escuela de
Ingeniería
Informática
Universidad de Oviedo



Universidad de
Oviedo

Fin

Manuel García Baldó – UO295497

Raúl Antuña Suárez – UO294202